

Assignment 9

Neural Networks, Perceptron & Deep Learning

Summer Internship — 27 June 2026

This assignment explores the foundational concepts of Neural Networks, the Perceptron model, and Deep Learning — all applied to the Titanic Survival dataset. Four sections are covered: Concept Review, Titanic Case Study, Real-World Applications, and a Reflection on AI thinking.

Assignment Overview

Part	Topic	Key Skill
A	Concept Review	Perceptron, Gradient Descent, Backpropagation
B	Titanic Case Study	Feature Selection, Data Flow, NN Diagram
C	Real-World Applications	Domain Matching with Neural Network Use Cases
D	Think & Reflect	Critical Thinking on AI and Deep Learning

Part A — Concept Review

Q1. Define Perceptron in Your Own Words

A Perceptron is the simplest unit of a neural network — inspired by a biological neuron. It takes multiple numerical inputs, multiplies each by a weight (importance), sums them up, adds a bias, and passes the result through an activation function to produce a binary output (0 or 1). Think of it as a single decision-maker: 'Given Age, Fare, and Class — did this Titanic passenger survive?'

Mathematical Formula

$$\text{output} = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b)$$

Python Code — Simple Perceptron

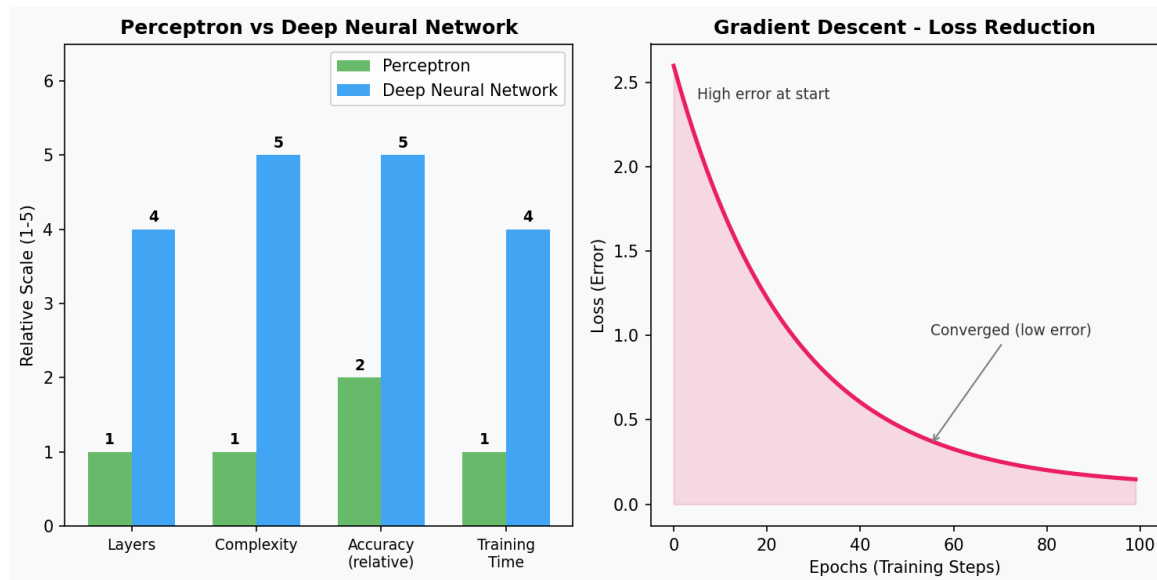
```
# Simple Perceptron in Python
import numpy as np

def perceptron(inputs, weights, bias):
    weighted_sum = np.dot(inputs, weights) + bias
    return 1 if weighted_sum >= 0 else 0 # Step activation

# Example: Titanic passenger [Age=25, Fare=50, Pclass=2]
inputs = np.array([25, 50, 2])
weights = np.array([-0.01, 0.02, -0.5]) # learned weights
bias = 0.3
output = perceptron(inputs, weights, bias)
print('Survived:', output) # 1 = Survived, 0 = Did Not
```

Q2. Three Key Differences: Perceptron vs. Deep Neural Network

Feature	Perceptron	Deep Neural Network
Layers	1 layer only (input to output)	Multiple hidden layers (deep)
Problem Type	Only linearly separable data	Complex, non-linear patterns
Learning	Simple weight update rule	Backpropagation + optimizers (Adam)
Use Case	Basic binary classification	Images, text, speech, survival prediction



Q3. What Does Gradient Descent Do in a Neural Network?

Gradient Descent is the learning algorithm that adjusts the weights of a neural network to minimize prediction error. During training, the network makes a prediction, calculates how wrong it was (the loss), then computes the gradient (direction of steepest increase in error), and moves the weights in the opposite direction — 'descending' the error hill step by step until the error is minimized.

Formula: $w = w - \alpha * (dL/dw)$ where α = learning rate, dL/dw = gradient

Q4. Why is Backpropagation Essential for Learning?

Backpropagation is the mechanism that makes gradient descent work in deep networks. Since a DNN has many layers, we need to know how much each weight contributed to the final error. Backpropagation computes this by applying the chain rule of calculus — passing the error signal backward through each layer and computing gradients layer by layer. Without it, deep networks could not learn: weights in early layers would never get updated.

Part B — Titanic Survival Case Study

Q1. Five Features Used to Predict Survival

Feature	Type	Why It Matters
Age	Numerical	Children were prioritised in lifeboat boarding
Fare	Numerical	Proxy for wealth; higher fare = better access to lifeboats
Sex	Categorical	Women had ~74% survival vs ~19% for men
Pclass	Ordinal (1/2/3)	1st class had lifeboats closer; 3rd class had physical barriers
Family Size	Numerical	Solo travellers or very large families fared worse

Q2. How Does Gradient Descent Help the Titanic Model?

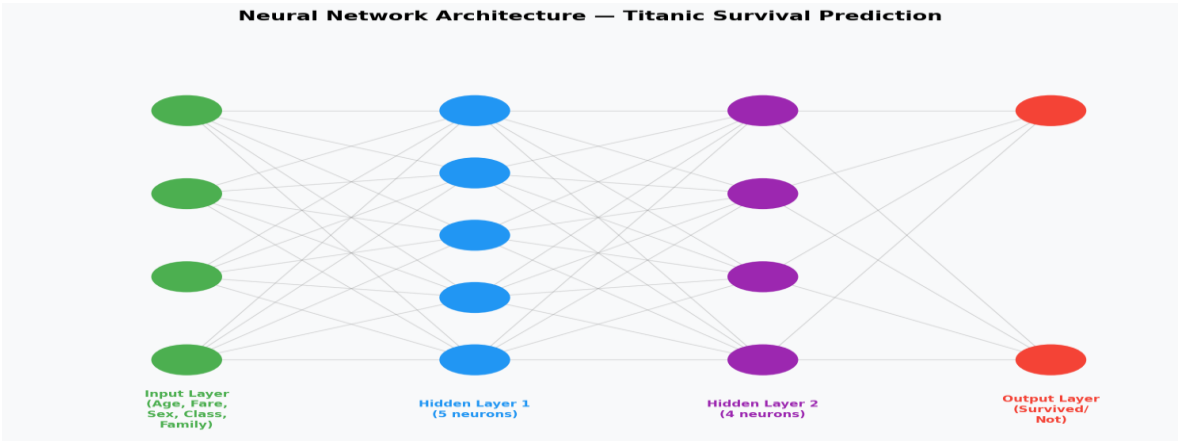
When training the Titanic neural network, initially the weights are random — the model might predict everyone survived. Gradient Descent iteratively adjusts the weights so the model's predictions improve with each epoch (training pass). For example, it learns that being female and in 1st class strongly increases survival probability — these patterns emerge purely from mathematical weight adjustments guided by the loss function.

```
# Gradient Descent Training Loop (Simplified)
for epoch in range(1000):
    y_pred = model.forward(X_train)          # Make prediction
    loss = cross_entropy(y_pred, y_train)    # Measure error
    gradients = model.backward(loss)         # Backpropagate
    model.weights -= learning_rate * gradients # Update weights
    if epoch % 100 == 0:
        print(f'Epoch {epoch}: Loss = {loss:.4f}')
```

Q3. What Patterns Might the Network Learn About Survival?

- Gender Signal: The network assigns a large positive weight to 'female', reflecting the 'women and children first' policy.
- Class Signal: Pclass=1 gets a strong positive weight; Pclass=3 gets a negative one — class strongly predicts survival.
- Age Effect: Young children (Age < 10) will get a positive bias. Middle-aged men get a negative bias.
- Fare Proxy: High fare passengers cluster with 1st class, reinforcing the wealth-survival relationship.
- Interaction Effects: Hidden layers capture combinations like 'young female in 2nd class' — better than any single feature alone.

Q4. Neural Network Architecture — Data Flow Diagram



Data flows left to right: raw passenger features enter the Input Layer, pass through two Hidden Layers where non-linear patterns are extracted via ReLU activation functions, and finally the Output Layer applies Sigmoid to produce a probability — above 0.5 means Survived.

Part C — Real-World Applications of Neural Networks

Domain-to-Application Matching (with Explanation)

Domain	Application	How Neural Networks Help
Healthcare	Tumor Detection	CNNs analyse MRI/X-ray images, learning visual patterns of malignant vs benign tumors. Accuracy rivals trained radiologists.
Finance	Market Trend Prediction	RNNs/LSTMs process time-series stock data, detecting temporal patterns and predicting price movements.
Space Science	Satellite Image Analysis	Deep CNNs classify terrain, detect weather systems, and track climate change from satellite imagery at scale.
Education	Adaptive Learning Systems	Neural networks track student performance, identify knowledge gaps, and personalise content delivery in real-time.

Full Neural Network Code — Titanic Survival Prediction

```
# Neural Network for Titanic - scikit-learn MLPClassifier
import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# Load and prepare data
df = pd.read_csv('titanic.csv')
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['FamilySize'] = df['SibSp'] + df['Parch']
df = df[['Age', 'Fare', 'Sex', 'Pclass', 'FamilySize', 'Survived']].dropna()

X = df.drop('Survived', axis=1)
y = df['Survived']

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build Neural Network (2 hidden layers: 64 and 32 neurons)
nn = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation='relu',      # ReLU in hidden layers
    solver='adam',         # Adam optimizer (smart gradient descent)
    max_iter=500,
    random_state=42
)
nn.fit(X_train, y_train)

# Evaluate
y_pred = nn.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.2%}')
print(classification_report(y_test, y_pred,
    target_names=['Did Not Survive', 'Survived']))
```

Part D — Think & Reflect

"From simple neurons to deep insights — AI is teaching machines to think."

Reflection Paragraph (5-6 Lines):

This statement captures the remarkable journey of artificial intelligence — from the humble Perceptron (a single neuron making a yes/no decision) to today's deep neural networks that diagnose diseases, predict stock markets, and drive autonomous vehicles. In the context of the Titanic, a simple perceptron might only learn 'women survive more.' But a deep neural network discovers why — the combination of gender, class, age, and family structure creates complex survival patterns that no single rule can capture. This mirrors human thinking: we don't decide with a single neuron — our brain uses billions of interconnected ones, each adding a tiny piece of understanding. AI is not replacing human thought; it is amplifying it — turning raw data into insight, and insight into action. The lesson from Assignment 9 is that learning is layered: just as neural networks build understanding layer by layer, so do we as students — each concept building on the last until deep understanding emerges.

Quick Summary Table

Key Concept	One-Line Summary
Perceptron	Single neuron — binary decision using weighted inputs + bias
Deep Neural Network	Many perceptrons stacked in layers — learns complex patterns
Gradient Descent	Moves weights downhill on the error surface to minimise loss
Backpropagation	Chain rule applied backward — teaches each layer how to improve
Activation Function	ReLU/Sigmoid — adds non-linearity so network can learn curves